

Pre-Partitioning Analysis I

- Specification requires a lot of effort to create
- Pre-partitioning analysis step is required to derive maximum value from the specification.
- Examine options costs and benefits in preparation for partitioning.
- Engineers often skip analysis and jump straight to partitioning, which is not good.
- Academics often linger too much on analysis, which is not good - death by analysis
- Analysis should reduce the time spent working on unattractive alternatives
- Gaining some idea of functional or algorithmic characteristics is worth doing.
- Implementation details should be avoided to the extent possible.

Pre-Partitioning Analysis II

- The flow rigid division into phases does not accurately reflect real design complexities.
- Designa base on a platform - some decisions are already made.
- Some implementation-level decisions will influencing the mental design process
- Depending on style of the specification, there are several ways in which analysis can be done:
 - Static Analysis
 - Platform-Based Analysis
 - Dynamic Analysis
 - Algorithmic Analysis

Static Analysis I

- If an executable specification is not available, the analysis must be performed statically.
- Usually informal
- Some of the techniques for static analysis include:
 - System complexity analysis
 - Reliability
 - Maintainability
 - “ility”...
- System complexity analysis is based on software engineering function point analysis
- It is not so important that the absolute error for static analysis, based on estimators derived either theoretically or from historical experience, be low, as that the **tradeoff decisions made based on the estimators be monotonic.**

Static Analysis II

- If analysis of choices A and B indicates that A is preferred to B, then **the actual system** design results for the criteria under analysis **match this** (A will prove to be better than B, within some reasonable envelope of uncertainty).
- However, even in the absence of a strong historical basis for the underlying predictor models, **simple analytical models** have proven to be of **value in making decisions** on lines of development.
- Static analysis examples are:
 - Traditional “ility” Analysis of Systems
 - Jira Story Points

Platform Based Analysis

- A reuse-intensive design style
- The inherent bias
- An integration oriented design approach emphasising **systematic reuse**, for developing complex products based upon platforms and compatible hardware and software virtual components, intended to reduce development risks, costs and time to market.
- Any derivative design based will have new functionality
- Xilinx Zynq and Vivado tools are example of platform.
- There is no technological fix for existing platform bias in the pre-partitioning analysis phase. There is only **design discipline**. In this case, each new requirement and associated specification and models should be analyzed **as if** one were starting with a **clean slate** for the design.

Dynamic Analysis I

- The pre-partitioning analysis is a process that further develops the system specification models.
- It also establishing additional or more detailed design constraints.
- **Dynamic analysis** affords us the opportunity to examine more accurate information only available from executable models, such as time-based performance (latency, throughput) and the influence of arbitration and scheduling policies.
- We can also derive **computational, communications, and power** burdens from the simulation of executable models.
- The main evaluation criteria **narrow** to some extent when we turn to **dynamic execution** or simulation of the specifications by executable models.

Dynamic Analysis II

- Here we look to characteristics that **depend on simulation**, such as more accurate performance estimation (delay, throughput, latency) and factors that have an impact on time-based performance, such as the causes of congestion, arbitration policies and priorities, and scheduling algorithms for execution of models.
- From the dynamic simulation of executable models at the pre-partitioning stage, we need to derive **important characteristics** of the underlying functionality that we will use in partitioning. These include:
 - Computation burden
 - Communication burden
 - Power burden

Dynamic Analysis III

- If a **specification model** for a particular function is in an **executable form** in C or C++ , but does not satisfy the constraints of any particular toolset and associated model of computation, as in an algorithmic analysis tool, then **host-based execution** and associated analysis is the current state-of-the-art approach for dynamic analysis.
- **Software profiling tools** can be used to produce estimates of computational burden when running many test sets and test scenarios.
- By weighting instructions with **estimates of power consumption** per instruction, **the power burden can be estimated**, assuming one has power consumption figures for memory accesses.
- Adding simple **monitoring functions** can be useful to track **the amount of data** and control communications that executing the functional model entails.

Dynamic Analysis IV

- It is of course important to remember that **the functional model** is at best an **imperfect** analog of the real implementation. Thus, all estimates derived from either static or dynamic analysis of the functional models that make up a system specification are **not necessarily totally accurate**, and should be used **with care** in making decisions. Monotonicity of estimates may be useful in making decisions despite inaccuracies.

Static vs Dynamic

- Static analysis is simpler.
- Static analysis does not require a model.
- Using dynamic analysis is superior in providing a better understanding of design constraints and specification details.

Algorithmic Analysis I

- The algorithmic tradeoffs spectrum
 - Time - computation load
 - Space
 - Power
 - Complexity
 - Time-to-market
 - Data transport amount
 - Bit error rate
 - ...
- The time domain - Big O notation
- The time domain - parallel vs serial
- Space - Data and control storage
- Speculative operations gain performance while wasting power
- Caching data - saves power

Algorithmic Analysis II

- Sometimes, the algorithm choice has a large impact on energy consumption
- The complexity domain: performance vs verifiability
- The ideal solution might be finished too late.
- Floating-point algorithm to fixed-point conversion
- Initial setting of fixed-point range and precision - less work post-partitioning
- Example: Filter Design

Downstream Use of Analysis Results I

- The case studies discussed earlier in the context of dynamic algorithmic analysis illustrated the possibilities in **using the models, specifications, and analysis outputs in downstream implementation and verification.**
- Among the possible downstream uses are:
 - Floating-point and fixed-point models and results used as **golden verification environments** for actual implementations in both hardware and software
 - **Algorithmic specifications** that can drive **software code generation** for execution on target processors and DSPs
 - **Algorithmic specifications** that can drive **hardware code generation** for RTL level synthesis
 - **Co-simulation** between **system-level simulation**, RTL simulation, and software execution of code on instruction set simulators
- **HLS** is an interesting target for **downstream use** of executable specification models from this stage of the process.

Downstream Use of Analysis Results II

- A number of experiments and attempts have been made to use **MATLAB and Simulink to drive hardware synthesis.**
- **Software code generation** represents a downstream use goal for specifications that is more **talked about than used in practice**, although the UML community has long held this up as one of its goals.

Specialized Future I

- The greatest success of pre-partitioning techniques is in **specific models of computation** and associated tools, such as dataflow analysis for signal and image processing applications.
- This leads to a thought that perhaps the future of ESL will see a distinct specification -> analysis -> partitioning -> implementation -> verification flow **replaced by a number of domain-specific integrated flows**, each dealing with the particular subsystems of an integrated product that fit into those design domains.
- If **generalized control design** could be done as successfully as signal processing design in wireless and wired communications; if **software implementations could be generated** from abstract models in UML or SysML; if **HLS takes off** and really solves the implementation problem for high-level specifications, then it may be **much harder to distinguish specific or distinct phases** in the ESL flow. Instead, we

could end up with the way to do dataflow algorithms, the way to do control algorithms, the way to implement software, and so forth.

- **For portions of a system specification** that fit into a particular design approach and satisfy the requirements of a particular model of computation (such as dataflow algorithms), the use of any **available ESL tools should be investigated and adopted.**
- Design groups should **evaluate** new specification notations and analysis **methods and tools as they emerge.** Keep a watch on new academic methods.